DD_TCK-01 - Ticketing and Case Management

Developer implementation guide: `DD_TCK-01-Ticketing_and_Case_Management-DEV_IMPL_GUIDE-v1.0.md`.

1. Purpose

TCK-01 defines the SOPHIX Ticketing and Case Management module.

The module owns customer and operational tickets: complaints, service issues, billing disputes that start in customer care, relocation/installation follow-up, support requests, internal escalations, and cases that may require field work.

This DD exists because Customer Service notes/interactions are not enough for case lifecycle management. A note records what happened. A ticket owns a trackable work item with priority, SLA, assignment, status, linked entities, attachments, escalation, and downstream actions such as creating a Work Order.

2. Scope

TCK owns:

- ticket/case lifecycle;
- ticket category and reason catalog;
- priority and SLA tracking;
- assignment to user/team/queue;
- ticket timeline;
- customer-visible and internal comments;
- attachments metadata;
- links to customer, account, subscription, invoice, payment, work order, equipment, and order;
- ticket-to-work-order request;
- synchronization of linked Work Order status back into the ticket timeline;
- ticket events for reporting, notifications, and Customer 360.

TCK does not own:

- customer/account master data. Owner: `ILM-CFG-01`.
- subscription state. Owner: `SUB-LM` and `SUB-WF`.
- invoices/payments/wallet/dunning. Owner: `BIL`.
- work order lifecycle. Owner: `WO-01`.
- equipment stock/serial state. Owner: `OSR`.
- notification delivery. Owner: `NOT-01`.
- binary object storage. Owner: `FOUNDATION_FILE_STORAGE`.

3. Runtime Actors

Actor	Responsibility
Backoffice Customer Care	Creates, updates, assigns, and resolves tickets.
Customer Self-Care	Creates customer-visible support requests and views ticket status where enabled.
Ticketing API	Owns ticket commands, validation, state transitions, and timeline writes.
Ticketing DB	Stores ticket tables and timeline.
ILM Customer Service	Provides customer/account/contact reference data.
SUB-LM/SUB-WF	Provides subscription state and operation references.
BIL	Provides invoice/payment/wallet/dunning references and dispute APIs.
WO	Owns Work Orders created from tickets and emits WO lifecycle events.

Actor	Responsibility
OSR	Provides equipment references when ticket concerns equipment.
NOT/ICN	Sends customer/staff notifications.
File Storage	Stores ticket attachments through authorized module APIs.

4. Architecture

```
flowchart LR
    BO[Backoffice / Customer Care] --> TCK[TCK Ticketing API]
    SELF[Customer Self-Care] --> TCK
    TCK --> ILM[ILM Customer API]
    TCK --> SUB[SUB-LM / SUB-WF APIs]
    TCK --> BIL[Billing APIs]
    TCK --> WO[WO-01 API]
    TCK --> OSR[OSR API]
    TCK --> NOT[NOT / ICN]
    TCK --> FILES[S3/MinIO via File API]
    WO -->|WorkOrder events| TCK
    TCK -->|Ticket events| KAFKA[Kafka]
```

5. Ticket Lifecycle

Status	Meaning	Terminal?
`DRAFT`	Ticket started but not submitted. Usually frontend draft; persisted only if configured.	No
`OPEN`	Ticket is accepted and waiting triage or work.	No
`TRIAGED`	Category/priority/owner confirmed.	No
`ASSIGNED`	Ticket assigned to a user/team/queue.	No
`WAITING_CUSTOMER`	Waiting for customer input.	No
`WAITING_INTERNAL`	Waiting for another SOPHIX module/team.	No
`WAITING_WORK_ORDER`	Linked Work Order is open and ticket waits for WO outcome.	No
`UNDER_REVIEW`	Supervisor/lead is reviewing resolution or escalation.	No
`RESOLVED`	Resolution recorded; may still be reopenable.	Yes
`CLOSED`	Case is closed after resolution window.	Yes
`CANCELLED`	Ticket cancelled as duplicate/error/customer withdrew.	Yes

Allowed high-level transitions:

```
OPEN -> TRIAGED -> ASSIGNED -> WAITING_* -> UNDER_REVIEW -> RESOLVED -> CLOSED
OPEN/TRIAGED/ASSIGNED/WAITING_* -> CANCELLED
RESOLVED -> OPEN only through reopen policy
```

6. Ticket Types

Type	Typical use	May create WO?
`TECHNICAL_SUPPORT`	Internet/TV/voice problem, outage check, onsite support request.	Yes
`INSTALLATION_FOLLOWUP`	Install missed, quality issue, incomplete install.	Yes
`RELOCATION_SUPPORT`	Help with move/shift issue.	Yes
`EQUIPMENT_ISSUE`	Faulty CPE, lost/damaged device, serial mismatch.	Yes
`BILLING_COMPLAINT`	Invoice/payment/wallet dispute or explanation.	Sometimes
`KYC_SUPPORT`	KYC correction or document issue.	No
`PACKAGE_OR_CATALOG_QUERY`	Package availability/offer explanation.	No
`GENERAL_INQUIRY`	Non-operational support request.	No
`INTERNAL_ESCALATION`	Staff-created internal follow-up.	Maybe

7. Tables

7.1 `ticket`

`ticket` stores the main case record. Use it to find, assign, prioritize, and resolve a customer or operational issue.

```
CREATE TABLE ticket (
  ticket_id              TEXT PRIMARY KEY,
  operator_code          TEXT NOT NULL,
  ticket_number          TEXT NOT NULL,
  customer_id            TEXT,
  account_id             TEXT,
  subscription_id        TEXT,
  status_code            TEXT NOT NULL,
  type_code              TEXT NOT NULL,
  category_code          TEXT NOT NULL,
  subcategory_code       TEXT,
  priority_code          TEXT NOT NULL,
  source_channel         TEXT NOT NULL,
  subject                TEXT NOT NULL,
  description            TEXT,
  customer_visible       BOOLEAN NOT NULL DEFAULT TRUE,
  assigned_queue_code    TEXT,
  assigned_team_id       TEXT,
  assigned_user_id       TEXT,
  sla_policy_code        TEXT,
  due_at                 TIMESTAMPTZ,
  first_response_due_at  TIMESTAMPTZ,
  first_response_at      TIMESTAMPTZ,
  resolved_at            TIMESTAMPTZ,
  closed_at              TIMESTAMPTZ,
  resolution_code        TEXT,
  resolution_summary     TEXT,
  reopened_count         INTEGER NOT NULL DEFAULT 0,
  current_wo_id          TEXT,
  idempotency_key        TEXT,
  request_hash           TEXT,
  created_by             TEXT NOT NULL,
  created_at             TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by             TEXT,
  updated_at             TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  version                BIGINT NOT NULL DEFAULT 0,
  UNIQUE (operator_code, ticket_number),
  UNIQUE (operator_code, idempotency_key)
);
```

Column meaning and examples:

Column	Purpose	Example	Meaning
`ticket_id`	Stable internal id.	`tck_01J2SUPPORT001`	Primary key.
`operator_code`	Operator scope.	`WIK`	Kenya first, future operators supported.
`ticket_number`	Human-facing reference.	`KE-TCK-2026-000001`	Shown to users/customers.
`customer_id`	Linked ILM customer.	`cust_01HX3K9M2P5`	No FK to ILM DB.
`subscription_id`	Linked subscription where relevant.	`sub_01HZ_KAMAU_FIBER`	Used for service-impact tickets.
`status_code`	Current lifecycle status.	`WAITING_WORK_ORDER`	Ticket waits for WO result.
`type_code`	Broad ticket type.	`TECHNICAL_SUPPORT`	Drives allowed actions.
`category_code`	Configured category.	`NO_INTERNET`	Used for routing/SLA/reporting.
`priority_code`	Priority.	`P2`	Drives due dates.
`source_channel`	Where ticket came from.	`CALL_CENTER`	Backoffice, self-care, social, field, etc.
`assigned_queue_code`	Queue owner.	`TECH_SUPPORT_L2`	Worklist routing.
`due_at`	SLA due date.	`2026-06-01T12:00:00Z`	Overdue if not resolved by then.
`current_wo_id`	Active linked WO if any.	`wo_01HZ_SUPPORT_01`	Reference to WO-01.
`idempotency_key`	Duplicate-submit protection.	`ui-req-01J2...`	Required for create commands.

7.2 `ticket_link`

`ticket_link` stores references to entities owned by other modules.

```
CREATE TABLE ticket_link (
```

```
  ticket_link_id TEXT PRIMARY KEY,
  ticket_id      TEXT NOT NULL REFERENCES ticket(ticket_id),
  entity_type    TEXT NOT NULL,
  entity_id      TEXT NOT NULL,
  link_role      TEXT NOT NULL,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (ticket_id, entity_type, entity_id, link_role)
);
```

Use this table when a ticket relates to multiple entities, such as a subscription, invoice, payment, work order, and equipment serial.

Column	Purpose	Example	Meaning
`entity_type`	Linked entity owner/type.	`WORK_ORDER`	Link to WO-01.
`entity_id`	External entity id.	`wo_01HZ_SUPPORT_01`	No cross-module FK.
`link_role`	Why linked.	`CREATED_FROM_TICKET`	Explains relationship.

Allowed `entity_type` values:

- `CUSTOMER`
- `ACCOUNT`
- `SUBSCRIPTION`
- `SUBSCRIPTION_OPERATION`
- `INVOICE`
- `PAYMENT`
- `WORK_ORDER`
- `EQUIPMENT_INSTANCE`
- `ORDER`
- `HOME_PASS`

7.3 `ticket_comment`

`ticket_comment` stores customer-visible and internal comments.

```
CREATE TABLE ticket_comment (
  comment_id       TEXT PRIMARY KEY,
  ticket_id        TEXT NOT NULL REFERENCES ticket(ticket_id),
  visibility       TEXT NOT NULL,
  body             TEXT NOT NULL,
  created_by       TEXT NOT NULL,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  superseded_by    TEXT
);
```

Column	Purpose	Example	Meaning
`visibility`	Who can see comment.	`INTERNAL`	Internal support note.
`body`	Comment text.	`Customer reports no signal after storm.`	Timeline note.
`superseded_by`	Replacement comment id if corrected.	`cmt_01J2...`	Audit-friendly correction.

7.4 `ticket_attachment`

`ticket_attachment` stores metadata for files attached to the ticket. Binary files are stored through the file storage foundation.

```
CREATE TABLE ticket_attachment (
  attachment_id    TEXT PRIMARY KEY,
  ticket_id        TEXT NOT NULL REFERENCES ticket(ticket_id),
  object_key       TEXT NOT NULL,
  file_name        TEXT NOT NULL,
  content_type     TEXT NOT NULL,
  size_bytes       BIGINT NOT NULL,
  visibility       TEXT NOT NULL,
  uploaded_by      TEXT NOT NULL,
  uploaded_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

7.5 `ticket_timeline_event`

`ticket_timeline_event` stores immutable ticket history.

```
CREATE TABLE ticket_timeline_event (
  event_id       TEXT PRIMARY KEY,
  ticket_id      TEXT NOT NULL REFERENCES ticket(ticket_id),
  event_type     TEXT NOT NULL,
  actor_id       TEXT NOT NULL,
  summary        TEXT NOT NULL,
  payload_json   JSONB NOT NULL DEFAULT '{}'::jsonb,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Timeline examples:

Event	Example summary
`TicketCreated`	Ticket opened from call center.
`TicketAssigned`	Assigned to queue `TECH_SUPPORT_L2`.
`WorkOrderCreatedFromTicket`	Work Order `wo_01HZ_SUPPORT_01` created.
`LinkedWorkOrderFinalized`	Work Order finalized with result `RESOLVED_ONSITE`.
`TicketResolved`	Ticket resolved as `SERVICE_RESTORED`.

7.6 `ticket_category_catalog`

Defines ticket categories, routing defaults, and whether the category may create a WO.

```
CREATE TABLE ticket_category_catalog (
  operator_code        TEXT NOT NULL,
  category_code        TEXT NOT NULL,
  display_name         TEXT NOT NULL,
  type_code            TEXT NOT NULL,
  default_priority     TEXT NOT NULL,
  default_queue_code   TEXT,
  default_sla_policy   TEXT,
  wo_allowed           BOOLEAN NOT NULL DEFAULT FALSE,
  default_wo_kind      TEXT,
  active               BOOLEAN NOT NULL DEFAULT TRUE,
  PRIMARY KEY (operator_code, category_code)
);
```

Sample data:

operator	category	type	priority	wo_allowed	default WO kind	Meaning
`WIK`	`NO_INTERNET`	`TECHNICAL_SUPPORT`	`P2`	true	`SUPPORT`	Can dispatch technician.
`WIK`	`BILLING_DISPUTE`	`BILLING_COMPLAINT`	`P3`	false	null	Routed to billing queue.
`WIK`	`INSTALL_INCOMPLETE`	`INSTALLATION_FOLLOWUP`	`P2`	true	`SUPPORT`	Follow-up WO may be created.

8. APIs

All APIs require JWT auth and role checks.

8.1 Ticket Create

```
POST /api/tickets
Idempotency-Key: ui-req-01J2SUPPORT001
Content-Type: application/json

{
  "operatorCode": "WIK",
  "customerId": "cust_01HZ_KAMAU",
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "typeCode": "TECHNICAL_SUPPORT",
  "categoryCode": "NO_INTERNET",
  "sourceChannel": "CALL_CENTER",
  "subject": "Customer has no internet",
  "description": "Customer reports LOS red light on ONT.",
  "links": [
    { "entityType": "EQUIPMENT_INSTANCE", "entityId": "eqp_ONT_42", "linkRole": "REPORTED_DEVICE" }
  ]
```

```
}
```

Response:

```
{
  "ticketId": "tck_01J2SUPPORT001",
  "ticketNumber": "KE-TCK-2026-000001",
  "statusCode": "OPEN",
  "priorityCode": "P2",
  "assignedQueueCode": "TECH_SUPPORT_L1",
  "dueAt": "2026-06-01T12:00:00Z"
}
```

8.2 Ticket Search

```
GET /api/tickets?operatorCode=WIK&customerId=cust_01HZ_KAMAU&status=OPEN,ASSIGNED&page=0&size=50
```

Supported filters:

- `operatorCode`
- `ticketNumber`
- `customerId`
- `accountId`
- `subscriptionId`
- `status`
- `typeCode`
- `categoryCode`
- `assignedQueueCode`
- `assignedUserId`
- `priorityCode`
- `createdFrom`
- `createdTo`
- `linkedEntityType`
- `linkedEntityId`

8.3 Ticket Detail

```
GET /api/tickets/{ticket_id}
```

Returns ticket, links, latest comments, attachments, and timeline summary.

8.4 Add Comment

```
POST /api/tickets/{ticket_id}/comments
Content-Type: application/json

{
  "visibility": "INTERNAL",
  "body": "Customer is available for technician visit tomorrow morning."
}
```

8.5 Assign Ticket

```
POST /api/tickets/{ticket_id}/assign
Content-Type: application/json

{
  "assignedQueueCode": "TECH_SUPPORT_L2",
  "assignedUserId": "agent_01J2MARY",
  "reason": "Needs technical triage."
}
```

8.6 Create Work Order From Ticket

This is the key integration with WO-01.

```
POST /api/tickets/{ticket_id}/work-orders
```

```
Idempotency-Key: ui-req-01J2WO001
Content-Type: application/json

{
  "woKind": "SUPPORT",
  "taskType": "SUPPORT",
  "preferredDate": "2026-06-02",
  "techRegionCode": "NRB-KAREN",
  "description": "No internet. Check ONT LOS and fiber drop.",
  "equipmentRefs": ["eqp_ONT_42"]
}
```

TCK validates the ticket and category, then calls WO-01:

```
POST /api/work-orders
Content-Type: application/json
```

TCK passes `callerContext`:

```
{
  "callerModule": "TCK",
  "callerTicketId": "tck_01J2SUPPORT001",
  "callerTicketNumber": "KE-TCK-2026-000001"
}
```

After WO creation succeeds, TCK:

1. updates ticket status to `WAITING_WORK_ORDER`;
2. sets `current_wo_id`;
3. inserts `ticket_link` to the WO;
4. writes timeline event `WorkOrderCreatedFromTicket`;
5. emits `TicketWorkOrderCreated`.

8.7 Resolve Ticket

```
POST /api/tickets/{ticket_id}/resolve
Content-Type: application/json

{
  "resolutionCode": "SERVICE_RESTORED",
  "resolutionSummary": "Technician replaced damaged drop cable. Service restored.",
  "customerVisible": true
}
```

8.8 Reopen Ticket

```
POST /api/tickets/{ticket_id}/reopen
Content-Type: application/json

{
  "reasonCode": "ISSUE_RECURRED",
  "comment": "Customer called again within 24 hours."
}
```

9. Work Order Integration

9.1 When Ticket May Create WO

TCK may create a WO only when:

- ticket status is not terminal;
- ticket category has `wo_allowed = true`;
- no active linked WO exists unless category allows multiple WOs;
- required customer/subscription/location/HomePass data is available;
- actor has a role allowed to request field dispatch;
- TCK can validate assignment references through WO/EM/Tech Region APIs where required.

9.2 WO Event Consumption

TCK consumes WO events:

Event	TCK behavior
`WorkOrderCreated`	Link WO if `callerContext.callerModule = TCK`.
`WorkOrderAssigned`	Add timeline event and optionally notify customer.
`WorkOrderPhaseFinalized`	Add timeline event for multi-phase work.
`WorkOrderFinalized`	Move ticket to `UNDER_REVIEW` or `RESOLVED` depending on category config and WO outcome.
`WorkOrderCancelled`	Move ticket back to `ASSIGNED` or `WAITING_INTERNAL` and require review.
`WorkOrderSupportCompleted`	Add support-specific resolution details.

Consumers must use inbox/idempotency rules from `FOUNDATION_KAFKA`.

10. Events

TCK publishes events through outbox.

Topic naming:

```
sophix.tck.ticket.<event>
```

Events:

Event	When emitted
`TicketCreated`	Ticket created.
`TicketAssigned`	Assignment changes.
`TicketCommentAdded`	Comment added.
`TicketWorkOrderCreated`	TCK successfully creates linked WO.
`TicketWaitingCustomer`	Ticket waits for customer.
`TicketWaitingWorkOrder`	Ticket waits for WO.
`TicketResolved`	Ticket resolved.
`TicketClosed`	Ticket closed.
`TicketReopened`	Ticket reopened.
`TicketCancelled`	Ticket cancelled.

Envelope example:

```
{
  "eventId": "tck:ticket:tck_01J2SUPPORT001:evt_01J2",
  "eventType": "TicketWorkOrderCreated",
  "aggregateType": "ticket",
  "aggregateId": "tck_01J2SUPPORT001",
  "operatorCode": "WIK",
  "occurredAt": "2026-06-01T09:12:22Z",
  "payload": {
    "ticketNumber": "KE-TCK-2026-000001",
    "workOrderId": "wo_01HZ_SUPPORT_01",
    "customerId": "cust_01HZ_KAMAU",
    "subscriptionId": "sub_01HZ_KAMAU_FIBER"
  }
}
```

11. Rules

Rule	Requirement
TCK-1	Ticket create requires idempotency key.
TCK-2	Ticket must link to at least one of customer, account, subscription, invoice, payment, order, work order, equipment, or explicit internal category.
TCK-3	Only configured categories may create WOs.
TCK-4	TCK must not directly update WO status. WO status changes only through WO APIs/events.
TCK-5	TCK must not directly read ILM/SUB/BIL/WO/OSR databases.
TCK-6	Work Order creation from ticket must be auditable and linked through `ticket_link`.

Rule	Requirement
TCK-7	Terminal tickets cannot create new WOs unless reopened first.
TCK-8	Customer-visible comments must not expose internal module names, private notes, or staff-only diagnostics.
TCK-9	Attachments must be stored through `FOUNDATION_FILE_STORAGE`; object keys must not contain PII.
TCK-10	WO events must be consumed idempotently.

12. Error Handling

Condition	HTTP	Error code
Ticket not found	404	`TICKET_NOT_FOUND`
Category not allowed to create WO	409	`TICKET_CATEGORY_WO_NOT_ALLOWED`
Active WO already linked	409	`ACTIVE_WORK_ORDER_ALREADY_LINKED`
Terminal ticket action attempted	409	`TICKET_TERMINAL_STATE`
Idempotency key reused with different payload	409	`IDEMPOTENCY_KEY_REUSED`
Linked entity not found	404	`LINKED_ENTITY_NOT_FOUND`
WO creation failed	502	`WORK_ORDER_CREATION_FAILED`
Unauthorized action	403	`TICKET_ACTION_FORBIDDEN`

13. Backoffice UX Requirements

Backoffice must provide:

- ticket queues by status, priority, SLA, category, queue, owner;
- ticket detail with linked customer/subscription/invoice/WO/equipment panels;
- create WO action only when allowed by category and role;
- timeline with comments, status changes, assignment changes, WO events;
- attachment viewer/uploader;
- customer-visible/internal comment distinction;
- reason/comment for cancellation and reopen;
- SLA overdue indicators;
- Customer 360 ticket panel.

14. Customer Self-Care Requirements

Customer Self-Care may expose:

- create support request;
- list customer-visible tickets;
- ticket detail with safe status labels;
- add customer-visible comment;
- upload approved attachments;
- view technician visit status if ticket created a WO.

Self-care must not expose:

- internal queue names;
- staff notes;
- Camunda incident details;
- internal WO failure diagnostics;

- private audit data.

15. Infrastructure Notes

TCK can start as a module inside the Backoffice/customer-service backend deployment if budget is constrained, but it must remain logically separate:

- own migrations;
- own tables;
- own API routes under `/api/tickets`;
- own outbox/inbox handlers;
- own metrics labels.

Recommended starting runtime:

Component	Starting shape
TCK API pods	2 pods, 2 vCPU, 4 GB RAM
TCK worker/outbox pods	2 pods, 1 vCPU, 2 GB RAM
Database	PostgreSQL Cluster B or C depending operational criticality; start Cluster B unless tickets become revenue-critical.
Kafka	Foundation Kafka
File storage	Foundation File Storage

Database placement recommendation:

- Start in Cluster B because tickets are operational/customer-care state.
- Do not put ticket tables in ILM, WO, or BIL databases.
- If ticket-driven field operations become mission-critical with strict RPO, revisit placement.

16. AWS Deployment

Use the standard SOPHIX AWS pattern:

- API/worker pods on EKS/ECS private subnets;
- RDS PostgreSQL in private data subnets;
- MSK for Kafka;
- S3 for attachments through application APIs;
- Secrets Manager/SSM for secrets;
- CloudWatch alarms for ticket create failures, WO creation failures, outbox backlog, inbox failures, SLA overdue volume.

17. Implementation Checklist

1. Create ticket tables and catalogs.
2. Seed WIK categories and SLA defaults.
3. Implement ticket create/search/detail/update APIs.
4. Implement comments and attachments metadata.
5. Implement assignment and queue movement.
6. Implement create-WO-from-ticket flow.
7. Implement WO event consumers.
8. Implement ticket outbox events.
9. Add Customer 360 ticket panel APIs.
10. Add Backoffice ticket queues.

11. Add self-care support request APIs where enabled.
12. Add metrics and audit logs.

18. DD References

DD	Relationship
`ILM-CFG-01`	Customer/account/contact/KYC reference data.
`SUB-LM-01`	Subscription references.
`SUB-WF-FRAMEWORK-01`	Ticket may be upstream reference for subscription operations.
`BIL-02-STA-01`	Billing disputes may create/link tickets.
`WO-01`	TCK creates and tracks linked Work Orders.
`OSR-INSTANCE-01`	Equipment references.
`NOT-01`	Customer/staff notifications.
`ICN-01`	Staff task/inbox notifications.
`FOUNDATION_FILE_STORAGE`	Ticket attachments.
`FOUNDATION_KAFKA`	TCK outbox/inbox and WO event consumption.